SANKET Platform — Comprehensive Project & Technical Report

Enterprise Multi-Industry Predictive Analytics & Supply Chain SaaS

Repository: Sanket-Project / Prisma	**Audit Date:** July 2026	**Status:** Advanced MVP / Operational Beta
Backend Stack: FastAPI, Python 3.12, PostgreSQL 16 + RLS	**ML Engine:** PyTorch, Chronos T5, LightGBM, OR-Tools	**Frontend:** React 18, Vite 5, TypeScript 5.6

Table of Contents

1. PROJECT OVERVIEW	Platform Identity, Value Proposition, Target Verticals & Operational Stage
2. FEATURES	Core System Capabilities, Specialized Features & In-Progress Roadmap Items
3. MODELS	ML/AI Forecasting Stack, Causal Engine, Optimization & Database Entities
4. FUNCTIONS & FUNCTIONALITIES	Core Backend Services, API Routers, ML Engine Modules & Connectors
5. USES	Realistic Industry Workflows & End-to-End Operational Scenarios
6. ARCHITECTURE	Full Tech Stack, Data Flow Pipelines, DevOps & Design Tradeoffs
7. COMPLETE PROJECT VIEW	Holistic Synthesis, Known Limitations & Immediate Strategic Next Steps

---

1. PROJECT OVERVIEW

Platform Identity & Problem Solved

SANKET (referenced in codebase manifests as *Prisma / Sanket-Project*) is a vertically integrated, multi-tenant SaaS platform delivering enterprise predictive analytics, demand forecasting, causal inference, and supply chain optimization. Enterprise supply chains struggle with extreme demand volatility, multi-echelon stockouts, costly overstocking capital drag, supplier lead-time variance, and stringent regulatory audit burdens. SANKET solves these challenges by combining advanced machine learning forecasters (foundation models, deep learning, gradient boosting, and statistical models) with mathematical supply-chain optimization, robust PostgreSQL row-level security (RLS), and industry-specific contextual profiles.

Target Users & Industry Verticals

The platform is specifically built for supply chain directors, demand planners, inventory managers, QA managers, and financial analysts across five target industry verticals:

Industry Vertical	Supply Chain Focus & Platform Solution	Compliance Level
Apparel & Fashion	Short product lifecycles, high seasonality, style/color/size SKU complexity. Solved via seasonal decomposition, Google Trends signals, and markdown optimization.	Standard
Consumer Electronics	Rapid product obsolescence, complex multi-tier BOMs, component lead-time volatility. Solved via component lead-time forecasting and automated shortage alerts.	Standard
Pharmaceuticals	Strict regulatory compliance, lot expiration risks, cold-chain temperature thresholds (-20°C to +8°C). Solved via FDA signal ingestion, batch quarantine/release, and immutable audit logs.	**GxP / 21 CFR Part 11**
Agriculture (Agrocenter)	Perishable goods shelf-life, crop yield volatility, macro weather dependencies. Solved via real-time Open-Meteo weather connectors and regional demand modeling.	Standard
Hardware	Project-based procurement, milestone-based delivery, long-lead supplier components. Solved via project BOM tracking and lead-time forecast modeling.	Standard

Current Project Stage

Based on direct codebase observation, SANKET is in an **Advanced MVP / Operational Beta** stage. The database layer (31+ tables, RLS enforcement, pgvector HNSW indexing, range partitioning), FastAPI backend, PyTorch ML engine, and React/Vite dashboard are fully functional. Recent core enhancements (July 2026) verified and resolved subscription entitlement guards (returning HTTP 402 for unsubscribed tenants) and workspace context token re-minting.

2. FEATURES

Core System Capabilities

- **PostgreSQL Row-Level Security (RLS):** Mandatory tenant isolation enforced at the DB kernel level via app.current_tenant_id and FORCE ROW LEVEL SECURITY on all tenant-scoped tables.
- **Multi-Vertical Workspace Switching:** Dynamic switching between Fashion, Electronics, Pharma, Agro, and Hardware via X-Industry-Code request header and validated JWT claims.
- **5-Stage Ensembled ML Forecasting:** Full pipeline: historical sales loading, intermittency classification, model pre-training, domain fitting, SLSQP convex ensemble optimization, and backtest reporting.
- **Zero-Shot Chronos Fallback:** Instant zero-shot demand forecasting via Amazon Chronos T5 (amazon/chronos-t5-small) when trained model artifacts are absent.
- **Pharma GxP & 21 CFR Part 11 Compliance:** Immutable audit logging (updates and deletes blocked by PostgreSQL SQL rules), batch lot quarantine/release tracking, cold-chain temperature validation, and QA signatures.
- **Supply Chain Optimization Suite:** Automated Safety Stock calculation, Economic Order Quantity (EOQ), Markdown pricing optimization, Multi-Echelon replenishment, and Reinforcement Learning (RL) replenishment.
- **External Signal Ingestion & Vector Clustering:** Real-time connectors for Weather (Open-Meteo), FRED macroeconomics, Google Trends, Reddit sentiment, Competitor prices, FDA regulatory notices, and 768-dim pgvector HNSW clustering.

Secondary & Specialized Features

- **Real-Time WebSockets:** Live sales streaming server (websocket.py) pushing notifications to connected client sessions.
- **Multi-Format Data Importer:** Async parsing and chunked DB insertion of CSV and Excel (.xlsx) sales data with schema validation.
- **E-Commerce Sync Connectors:** Direct pollers and sync handlers for Shopify REST/GraphQL and WooCommerce endpoints.
- **Direct SQL Data Connector:** Direct integration capabilities for external MySQL and PostgreSQL source databases.
- **Financial & What-If Scenario Analysis:** Simulation engines evaluating revenue exposure, margin impact, and stockout penalties.
- **Razorpay Subscription Billing:** Multi-tier billing management (Growth, Scale, Enterprise) supporting checkout verification, webhooks, and tier usage tracking.

In-Progress & Roadmap Items

- *Razorpay Live Production Execution:* Execution of live key checkout verification in production payment mode.
- *Firebase Token Custom Claims Sync:* Real-time custom claims sync on dynamic industry change in production Firebase auth mode.
- *Enterprise Connector Expansion:* Expanding built-in connectors to match the complete 30+ advertised enterprise catalog.

3. MODELS

ML & AI Forecasting Models Stack

SANKET incorporates a multi-tiered ensemble of statistical, gradient boosted, deep learning, foundation, and causal models located in backend/ml/sanket_ml/models/:

Model Category	Model Name & Module Path	Purpose & Implementation Details
Foundation Models	• Chronos (foundation/chronos.py) • Lag-Llama (foundation/lag_llama.py) • Moirai (foundation/moirai.py) • TimesFM (foundation/timesfm.py)	Pre-trained time-series foundation models. Amazon Chronos T5 Small provides instant zero-shot baseline forecasts without prior model training.
Deep Learning	• DeepAR (deep/deepar.py) • N-HiTS (deep/nhits.py) • TFT (deep/tft.py)	PyTorch Lightning & Darts deep forecasters capturing non-linear temporal interactions, static covariates, and multi-horizon outputs.
Gradient Boosting	LightGBM Forecaster (gradient_boost/lightgbm_forecaster.py)	LightGBM wrapped with mlforecast for high-speed tabular feature engineering (lags, rolling stats, calendar features).
Statistical / Intermittent	• Croston (statistical/croston.py) • ETS (statistical/ets.py) • Naive (statistical/naive.py) • ADIDA (adida.py)	StatsForecast baselines. Croston and ADIDA specifically handle sparse, intermittent, and low-volume SKU demand profiles.
Ensemble & Meta	• Stacked Ensemble (ensemble.py) • Meta Learner (meta_learner.py)	Solves SLSQP convex optimization to minimize pinball loss across holdout windows, producing optimal convex model weights.
Causal & Elasticity	• DoWhy Causal (causal/dowhy_pipeline.py) • Uplift Model (causal/uplift.py) • Price Elasticity (price_elasticity.py)	EconML & DoWhy causal graphs estimating treatment effects of marketing promotions, price adjustments, and market shocks.
Optimization	• EOQ (optimization/eoq.py) • Markdown (optimization/markdown.py) • Replenishment (optimization/replenishment.py) • RL Planner (optimization/rl_replenishment.py)	Google OR-Tools and Ray RLlib algorithms solving multi-echelon inventory replenishment, safety stock, and markdown pricing.

Key Database Entities & Schemas

The relational data model defined in backend/sql/002_schema.sql and Alembic migrations comprises 31+ core tables:

- **tenants & users:** Root tenancy entity (slug, tier, active_industry, max_skus) and user accounts with Argon2 password hashes, Firebase UIDs, and 4-tier roles.
- **industries & industry_profiles:** Global industry lookup (default horizon, required signals, model lists, audit levels) and tenant-specific profile overrides.
- **products & skus:** Product catalog and granular SKU records (gtin, lead_time_days, moq, safety_stock, reorder_point, cost, price, attributes JSONB).
- **historical_sales (Partitioned):** Range-partitioned quarterly (2022–2027) storing sales transactions, channel, region, units sold, revenue, and in_stock flags.
- **pharma_batches (GxP):** Pharmaceutical lot tracking (lot_number, ndc_code, expiry_date, gxp_status, cold_chain_required, storage_temp min/max, qa_released_by).
- **external_signals & signal_clusters:** Market signals (sentiment, processed values) and pgvector 768-dim HNSW vector embedding centroid clusters.
- **forecast_runs & forecast_results (Partitioned):** Execution run metadata and quarterly range-partitioned percentile demand predictions (p10, p50, p90).
- **audit_log:** Immutable 21 CFR Part 11 append-only audit trail protected by PostgreSQL NO-UPDATE and NO-DELETE rules.

4. FUNCTIONS & FUNCTIONALITIES

Core Modules & Service Layer

Module Path	Key Functions & Classes	Core Responsibility
backend/app/core/database.py	• Database.session(tenant_id) • session_no_rls() • assert_tenant_isolation_enforced()	Manages async SQLAlchemy engine, sets app.current_tenant_id per session, and enforces RLS security assertions on startup.
backend/app/core/rbac.py	• require_role() • require_role_db() • require_admin()	Enforces 4-role RBAC authorization (owner, admin, analyst, viewer) with DB re-verification for state-modifying operations.
backend/app/services/hybrid_forecast.py	• HybridForecastService.generate_forecast() • _call_ml_api()	Orchestrates forecast generation by calling the ML API or invoking zero-shot Chronos, storing results to forecast_results.
backend/app/services/billing/subscription_manager.py	• SubscriptionManager.create_subscription() • check_entitlement()	Handles Razorpay subscription lifecycle, plan upgrades, tier SKU limit checks, and active subscription verification.
backend/app/services/external_signals/signal_pipeline.py	• SignalPipeline.run_ingestion() • _cluster_signals()	Executes multi-connector signal ingestion, processes sentiment scores, and builds pgvector HNSW embeddings.
backend/app/services/integrations/file_import.py	• FileImportService.process_file() • _validate_rows()	Parses uploaded CSV/XLSX sales feeds, validates column schemas, and performs bulk upserts into historical_sales.
backend/app/workers/forecast_worker.py	• WorkerSettings • run_hybrid_forecast_job()	Durable background job worker powered by arq and Redis for non-blocking asynchronous forecasting tasks.

API Routers & Background Processing

- **Auth & Tenant Routers (auth.py, industry_router.py):** Login, signup, JWT issuance, MFA verification, and /industry/activate workspace switching.
- **Forecast & Analytics Routers (forecasts.py, hybrid_forecast.py, sales_analytics.py):** Submitting forecast runs, retrieving P10/P50/P90 time series, and analyzing sales metrics.
- **Industry Vertical Routers (fashion.py, electronics.py, pharma.py, agrocenter.py, hardware.py):** Specialized endpoints returning vertical KPIs, batch release actions, and BOM alerts.
- **Billing & Subscription Router (billing.py):** Plan listings, checkout sessions, Razorpay webhooks, and usage metrics.
- **Background Workers:** arq worker executing background forecast jobs; background pollers for e-commerce sync (Shopify, WooCommerce) and external signals.

5. USES

End-to-End Operational Scenarios

Scenario A: Fashion Retailer Markdown & Seasonality Optimization

1. *Data Ingestion:* Demand planner uploads 2 years of seasonal sales history via file_import.py.
2. *Forecasting:* Platform triggers 5-stage ML pipeline. StackedEnsemble fits LightGBM + Croston + DeepAR on seasonal apparel panel.
3. *Signal Fusion:* Google Trends connector (trends_connector.py) detects declining trend for summer apparel styles.
4. *Optimization:* System invokes markdown.py to generate an optimal weekly price reduction schedule that maximizes gross margin while liquidating inventory before autumn season launch.
5. *Execution:* Planner reviews P10/P50/P90 forecasts on FashionDashboard.tsx and pushes markdown prices to e-commerce channels.

Scenario B: Pharmaceutical GxP Cold-Chain & Expiry Management

1. *Tenant Onboarding:* Customer activates Pharmaceutical workspace (pharma). Backend sets audit level to gxp and enforces 52-week horizon.
2. *Batch Registration:* Production line registers new vaccine batch (lot #, NDC code, manufacture/expiry dates, storage range: -20°C to -15°C).
3. *Regulatory Monitoring:* fda_connector.py scans FDA enforcement reports for competing drug recall notices.
4. *QA Release Workflow:* QA Lead inspects batch temp logs and signs digital release (qa_released_by). Backend records append-only event in audit_log.
5. *Shortage Prevention:* Zero-shot Chronos forecaster detects impending regional clinic stockout 12 weeks out, prompting automatic replenishment.

6. ARCHITECTURE

Full Technology Stack Summary

Layer	Technologies & Frameworks	Role in System
Frontend	Vite 5, React 18, TypeScript 5.6, Tailwind CSS, TanStack Query, Zustand, Recharts	Single Page Application dashboard delivering responsive charts, workspace switching, and real-time updates.
Backend API	FastAPI 0.115, Python 3.12, Uvicorn, Pydantic v2, SQLAlchemy 2.0 (asyncio), asyncpg	Asynchronous REST API, RBAC, tenant context middleware, billing, and integrations.
ML Engine	PyTorch 2.4, LightGBM 4.5, Amazon Chronos 1.4, StatsForecast, OR-Tools 9.11, DoWhy	Dedicated ML inference API (:8001), 5-stage training pipeline, causal graphs, zero-shot forecaster.
Database	PostgreSQL 16, pgvector (0.7+), Row-Level Security (RLS), Declarative Partitioning	Primary datastore enforcing strict tenant isolation, 768-dim vector search, quarterly range partitions.
Cache / Worker	Redis 7, arq (async Redis job queue)	Session caching, rate limiting, and durable background hybrid forecast job queue execution.
DevOps / Infra	Docker Compose, Kubernetes (Kustomize), Nginx, Prometheus 2.54, Grafana 11.2	Container orchestration, ingress reverse proxying, metrics collection, and operational dashboards.

System Architecture & Data Flow Diagram

```
+-------------------------------------------------------------------------------+ | CLIENT LAYER (Nginx Ingress / TLS) | +-------------------------------------------------------------------------------+ | / (Static Assets) | /api/v1 (REST / WS) | /ml (Inference) v v v +------------------+ +------------------+ +------------------+ | React SPA | | FastAPI Backend | | ML Inference API | | Vite + TS | | (port 8000) |------------>| (port 8001) | | Port 8080 | | • Tenant RLS | HTTPX | • Chronos T5 | +------------------+ | • Industry Router| Bearer | • Joblib Ensembles| | • GxP Batch Audit| | • OR-Tools Supply| +------------------+ +------------------+ | | +--------------------+--------------------------------+ | | v v +----------------------------------+ +----------------------------------+ | PostgreSQL 16 + RLS | | Redis 7 + arq Queue | | • pgvector 768-dim HNSW Indexing | | • Session & Rate-Limit Cache | | • Range-Partitioned Sales & Results| | • Asynchronous Background Worker | | • Immutable Audit Trail (SQL Rules| | • Webhook Delivery & PubSub | +----------------------------------+ +----------------------------------+
```

Key Architectural Design Decisions & Tradeoffs

1. **Shared Database Schema with PostgreSQL RLS vs. Database-per-Tenant:**
Decision: Utilize PostgreSQL Row-Level Security (RLS) driven by app.current_tenant_id.
Tradeoff: Drastically reduces database infrastructure cost and simplifies schema migrations across hundreds of tenants. The tradeoff is strict reliance on session initialization; mitigated by startup role assertions (refusing to boot if the DB role bypasses RLS) and FORCE RLS on all tables.

2. **Decoupled Microservice ML API vs. Monolithic API:**
Decision: Separate the ML inference stack into a dedicated FastAPI service on port 8001.
Tradeoff: Prevents heavy PyTorch/Transformers CPU/GPU memory loads and long inference compute from blocking the web API event

loop. The tradeoff is inter-service HTTP latency and model artifact file sync requirements.

7. COMPLETE PROJECT VIEW

Holistic Synthesis & Project Maturity

SANKET represents a highly sophisticated, production-grade engineering foundation designed for enterprise supply chain analytics. The core data tier, multi-tenant isolation model, 5-stage ML forecasting engine, and specialized vertical contexts demonstrate exceptional architectural rigor. With recent fixes to subscription entitlement enforcement and workspace token re-minting, the backend core is ready for cloud deployment.

Known Limitations & Verified Gaps

- **Razorpay Payment Gateway Verification:** Subscription revenue-path models and entitlement guards (HTTP 402) are functional, but live payment checkouts require configuration of production Razorpay API keys.
- **Firebase Auth Custom Claims in Prod Mode:** Token claims refresh correctly in dev-login fallback mode; production Firebase mode requires cloud project verification for custom claims synchronization on workspace change.
- **Connector Integration Catalog Expansion:** Built-in connectors (Shopify, WooCommerce, SQL Source, File Importer, 7 External Signal Connectors) are complete; expanding to the full 30+ enterprise connector catalog remains an ongoing phase.

Strategic Next Steps & Recommendations

1. *Execute Live Payment Checkout Test:* Provide sandbox Razorpay API keys to test end-to-end checkout, invoice generation, and webhook handling.
2. *Deploy Staging Environment to Cloud:* Provision managed PostgreSQL 16 (Cloud SQL / RDS) with pgvector and deploy backend/ml services to Cloud Run / GKE using existing Kustomize manifests.
3. *Expand Automated Frontend E2E Tests:* Extend Playwright test coverage across paywall gates, workspace switching, and GxP batch release workflows.

Report compiled automatically from codebase audit of Sanket-Project / Prisma workspace.